Solution: Product of Array Except Self

Problem Statement

Given an array of integers, return a new array where each element at index `i` of the new array is the product of all the numbers in the original array except the one at `i`. You must solve this problem without using division.

Example Generation

1.
   - **Input**: `[2, 3, 4, 5]`
   - **Expected Output**: `[60, 40, 30, 24]`
   - **Justification**: For the first element: `3*4*5 = 60`, for the second element: `2*4*5 = 40`, for the third element: `2*3*5 = 30`, and for the fourth element: `2*3*4 = 24`.
2.
   - **Input**: `[1, 1, 1, 1]`
   - **Expected Output**: `[1, 1, 1, 1]`
   - **Justification**: Every element is 1, so the product of all other numbers for each index is also 1.
3.
   - **Input**: `[10, 20, 30, 40]`
   - **Expected Output**: `[24000, 12000, 8000, 6000]`
   - **Justification**: For the first element: `20*30*40 = 24000`, for the second element: `10*30*40 = 12000`, for the third element: `10*20*40 = 8000`, and for the fourth element: `10*20*30 = 6000`.

Solution

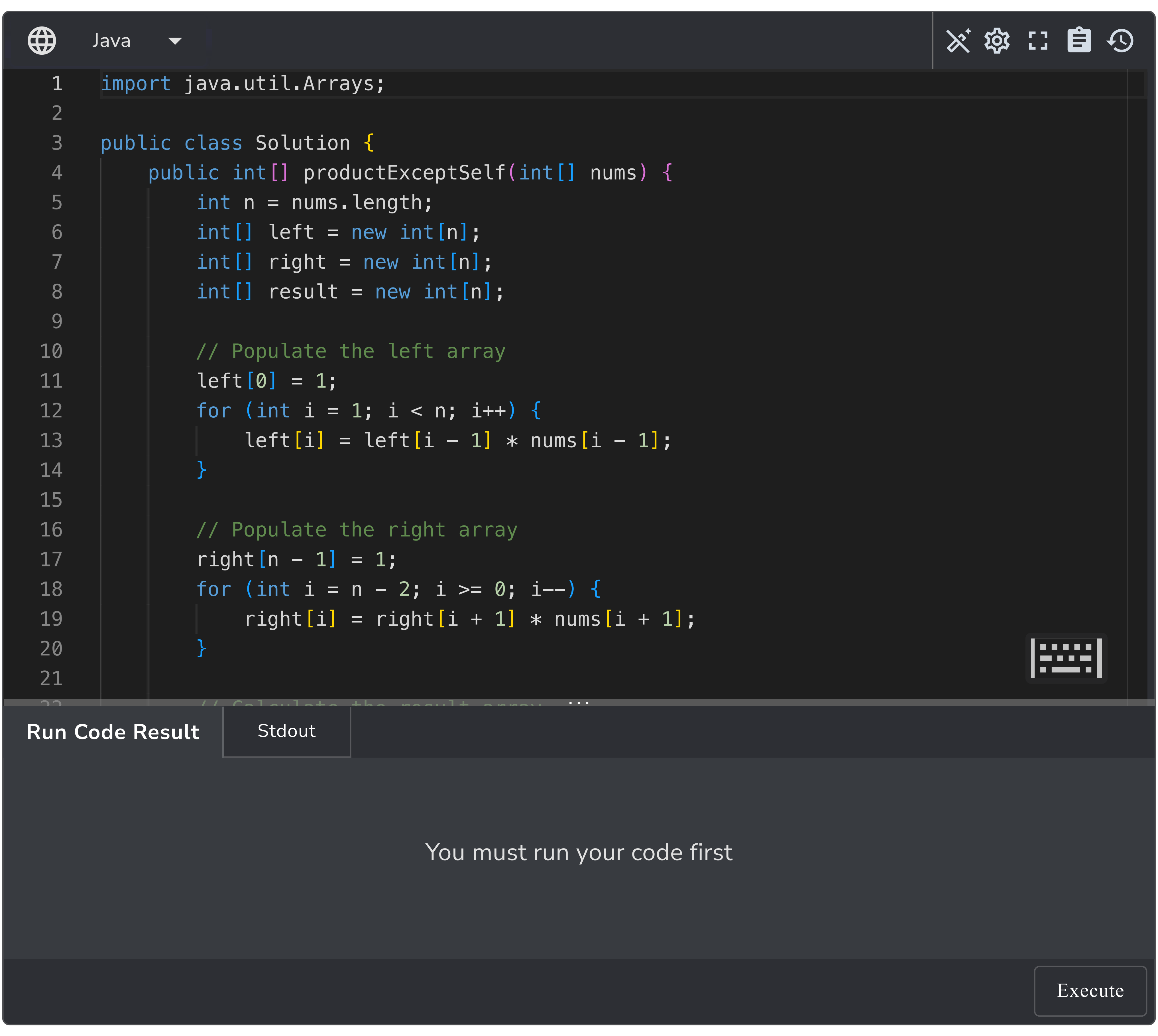
- **Initialize Two Arrays**:
  - Start by initializing two arrays, `left` and `right`. The `left` array will hold the product of all numbers to the left of index `i`, and the `right` array will hold the product of all numbers to the right of index `i`.
- **Populate the Left Array**:
  - The first element of the `left` array will always be `1` because there are no numbers to the left of the first element.
  - For the remaining elements, each value in the `left` array is the product of its previous value and the corresponding value in the input array.
- **Populate the Right Array**:
  - Similarly, the last element of the `right` array will always be `1` because there are no numbers to the right of the last element.
  - For the remaining elements, each value in the `right` array is the product of its next value and the corresponding value in the input array.
- **Calculate the Result**:
  - For each index `i`, the value in the result array will be the product of `left[i]` and `right[i]`.

Algorithm Walkthrough

Using the input `[2, 3, 4, 5]`:

1. Initialize `left` and `right` arrays with the same length as the input array and fill them with `1`.
2. Populate the `left` array:
   - `left[0] = 1`
   - `left[1] = left[0] * input[0] = 2`
   - `left[2] = left[1] * input[1] = 6`
   - `left[3] = left[2] * input[2] = 24`
3. Populate the `right` array:
   - `right[3] = 1`
   - `right[2] = right[3] * input[3] = 5`
   - `right[1] = right[2] * input[2] = 20`
   - `right[0] = right[1] * input[1] = 60`
4. Calculate the result:
   - `result[0] = left[0] * right[0] = 60`
   - `result[1] = left[1] * right[1] = 40`
   - `result[2] = left[2] * right[2] = 30`
   - `result[3] = left[3] * right[3] = 24`

Code



```java
import java.util.Arrays;

public class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] left = new int[n];
        int[] right = new int[n];
        int[] result = new int[n];

        // Populate the left array
        left[0] = 1;
        for (int i = 1; i < n; i++) {
            left[i] = left[i - 1] * nums[i - 1];
        }

        // Populate the right array
        right[n - 1] = 1;
        for (int i = n - 2; i >= 0; i--) {
            right[i] = right[i + 1] * nums[i + 1];
        }

```

Complexity Analysis:

- **Time Complexity**: O(n). We traverse the input array three times, so the time complexity is linear.
- **Space Complexity**: O(n). We use three additional arrays (`left`, `right`, and `result`).

← Previous
Product Of Array Except Self (Medium)

Next →
Maximum Product Subarray (Medium)

☐ Mark as Completed